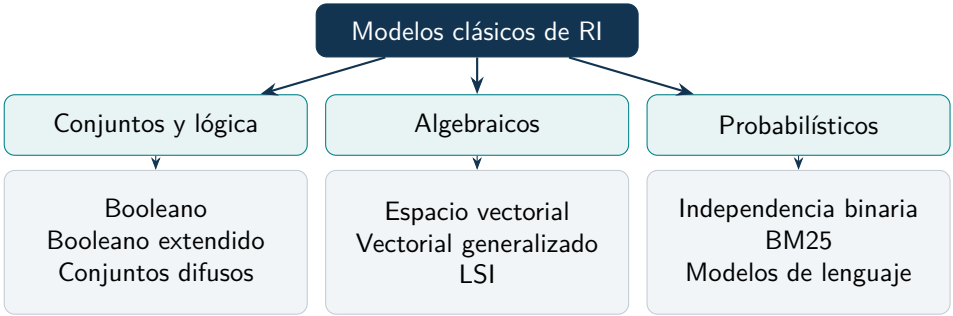
Embeddings y recuperación semántica

Iván Carrera

Escuela Politécnica Nacional
Recuperación de Información

Taxonomía clásica de los modelos de recuperación



Fuente: Síntesis didáctica. Baeza-Yates y Ribeiro-Neto, cap. 2; Manning et al., caps. 1, 6, 11, 12 y 18.

¿Dónde encaja esta clase?

Punto de partida

Ya representamos documentos y consultas, calculamos scores y obtenemos rankings.

1. **Hoy:** aprender representaciones densas y construir un ranking semántico.
2. **Siguiente clase:** evaluar TF-IDF, BM25 y embeddings sobre las mismas consultas y juicios de relevancia.
3. **Después:** estudiar búsqueda vectorial, índices y escalabilidad.

Para discutir

¿Qué cambia si consulta y documento expresan la misma idea con palabras diferentes?

Objetivos de aprendizaje

Al terminar la clase, podrán:

- Explicar qué es un embedding y distinguirlo de un vector TF-IDF.
- Diferenciar representaciones de palabras, tokens contextualizados y textos.
- Describir cómo el entrenamiento organiza el espacio vectorial.
- Calcular scores y construir un ranking con embeddings.
- Explicar un bi-encoder e identificar limitaciones de la recuperación semántica.

Pregunta transversal

¿Qué relaciones conserva la representación y cuáles puede perder?

Un caso que usaremos durante toda la clase

Consulta: «¿Cómo reparar el motor de un automóvil?»

ID	Documento
d_1	Guía para diagnosticar y corregir averías del propulsor de un vehículo.
d_2	Catálogo de automóviles: motores, colores y precios de modelos nuevos.
d_3	Cómo reparar una bicicleta: frenos, cadena y neumáticos.

Para discutir

¿Cuál responde mejor? ¿Cuál comparte más palabras? ¿Obtendremos siempre el mismo orden?

La coincidencia léxica aporta evidencia útil

TF-IDF y BM25 aprovechan:

- Presencia de términos de la consulta.
- Frecuencia y discriminación de términos.
- En BM25, saturación y longitud.

Sin expansión u otros mecanismos, una paráfrasis sin términos compartidos puede recibir un score muy bajo.



Fuente: Manning et al., cap. 6 y sección 11.4.3.

Recordatorio: el espacio vectorial con TF-IDF

Con un vocabulario $V = \{t_1, \ldots, t_{|V|}\}$:

$$\mathbf{x}_d = (w_{1,d}, w_{2,d}, \ldots, w_{|V|,d}), \qquad w_{j,d} = \mathrm{tf}(t_j, d)\,\mathrm{idf}(t_j).$$

- Cada dimensión corresponde a un término conocido.
- La dimensión depende del vocabulario de la colección.
- Cada documento utiliza una pequeña parte del vocabulario: el vector suele ser **disperso**.

Lo que conservaremos

Consultas y documentos seguirán siendo vectores. Seguiremos calculando similitudes y rankings.

Fuente: Manning et al., secciones 6.2 y 6.3.

¿Qué es un embedding de texto?

Un encoder f_θ transforma un texto x en un vector de dimensión fija m:

$$\mathbf{z}_x = f_\theta(x) \in \mathbb{R}^m.$$

- θ: parámetros aprendidos del modelo.
- m: dimensión de salida, fijada por la configuración del modelo.
- $\mathbf{z}_x$: representación numérica del texto.

Interpretación

El entrenamiento busca que ciertas relaciones entre textos se reflejen en sus representaciones y scores.

La relación aprendida depende de los datos y del objetivo: contexto lingüístico, similitud, relevancia u otra tarea.

Fuente: Lin, Nogueira y Yates, secciones 5.1 y 5.4.

Vectores dispersos y densos

	TF-IDF	**Embedding denso**
Dimensiones	Una por término del vocabulario	Número fijo definido por el modelo
Coordenadas	Pesos asociados a términos	Valores aprendidos conjuntamente
Ceros	Habitualmente muchos	Habitualmente pocos
Interpretación	Cada eje tiene un término identificable	Un eje aislado suele carecer de interpretación directa
Relaciones	Coincidencia de términos ponderados	Relaciones favorecidas por el entrenamiento

Dos criterios diferentes

Disperso/denso describe la representación. Clásico/neuronal describe otro aspecto: también existen representaciones dispersas neuronales.

¿Qué significa estar cerca en el espacio?

Con un modelo adecuado, esperamos mayor score para:

- Una consulta y un pasaje que la responde.
- Dos paráfrasis, si se entrenó para similitud textual.

La cercanía puede expresar relación temática sin que exista una respuesta útil.



Word embeddings: un vector por palabra

Word2Vec y GloVe aprenden representaciones a partir de patrones de coocurrencia.

- Palabras usadas en contextos relacionados pueden obtener representaciones próximas.
- El significado se distribuye entre muchas coordenadas.
- En su forma estática, una palabra conserva el mismo vector en distintos enunciados.

Una limitación: polisemia

«Me senté en el **banco** del parque.»
«Solicité un préstamo en el **banco**.»

Una única representación estática mezcla usos diferentes de la palabra.

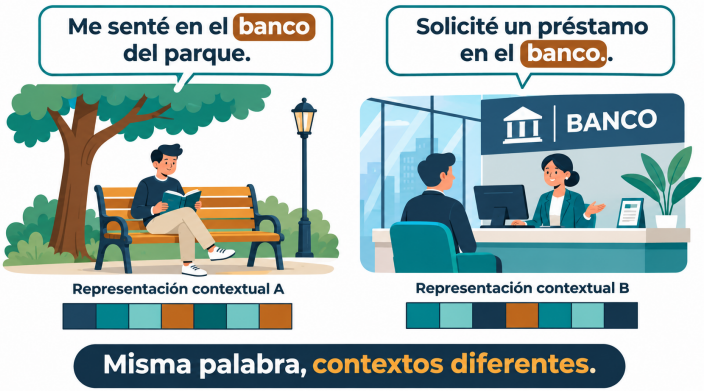
Fuente: Lin et al., secciones 1.2.3 y 5.3, antecedentes de representaciones distribuidas.

De la palabra al token contextualizado

Un modelo contextual depende del enunciado:

$$\mathbf{h}_i = F_\theta(t_1, \ldots, t_n)_i.$$

- t_i: token de entrada.
- $\mathbf{h}_i$: vector contextual de salida.
- Una palabra puede dividirse en subpalabras.



Fuente: Lin et al., sección 2. Ilustración conceptual: las celdas no son valores medidos.

Transformers: el recorrido conceptual

1. **Tokenización:** convertir el texto en una secuencia de identificadores.
2. **Entrada:** combinar embeddings de tokens e información de posición, según la arquitectura.
3. **Contextualización:** la autoatención combina información entre posiciones.
4. **Salida:** obtener un vector contextual por token.

Idea central de la atención

La representación de una posición incorpora información de otras posiciones mediante pesos calculados por el modelo.

Para recuperar textos completos necesitamos además decidir cómo formar su representación y cómo entrenarla.

Fuente: Lin et al., sección 2.

Del conjunto de tokens a un vector de texto

El *pooling* agrega las representaciones contextualizadas.

$$\mathbf{z} = \frac{\sum_{i=1}^{n} a_i \mathbf{h}_i}{\sum_{i=1}^{n} a_i} \qquad \text{(mean pooling en una configuración habitual).}$$

- $a_i \in \{0, 1\}$ identifica posiciones incluidas en el promedio, según la máscara.
- Los tokens de relleno deben excluirse.
- Otras configuraciones usan un token especial o max pooling.

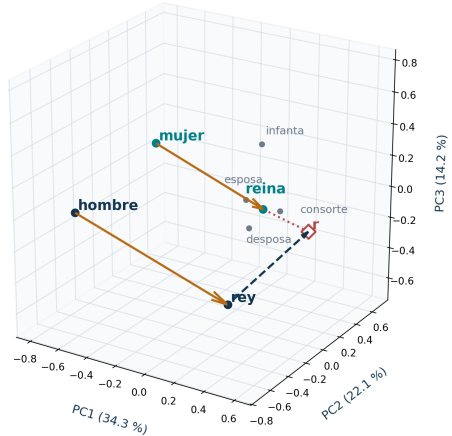
El pooling por sí solo no asegura buena recuperación

Importan el modelo, el entrenamiento y la compatibilidad entre representación y función de scoring.

Fuente: Lin et al., sección 5.4.1, Sentence-BERT.

Analogía semántica: búsqueda real y PCA en 3D

$$\mathbf{r} = \widehat{\mathbf{v}}_{rey} - \widehat{\mathbf{v}}_{hombre} + \widehat{\mathbf{v}}_{mujer}$$



MOST_SIMILAR · TOP 5

fastText · español · 300 dimensiones
Vocabulario: 985 667 palabras

Posición	Palabra	Coseno
1	**reina**	0.6612
2	consorte	0.5856
3	desposa	0.5714
4	infanta	0.5498
5	esposa	0.5492

«reina»: posición 1
coseno = 0.6612

- hombre → rey / mujer → reina
- mujer − hombre, desde rey
- r y diferencia respecto a reina
- Otros vecinos del top 5

Búsqueda exhaustiva en 300D. PCA ajustado a 8 palabras (las cuatro de la analogía y el top 5). Varianza conservada: 70.7%.

Vectores de entrada normalizados L2, como en Gensim. Los scores se calculan antes del PCA; las tres palabras de entrada se excluyen.

Fuente: fastText Wikipedia · wiki.es.vec. Gensim 4.4.0. Proyección únicamente para visualizar.

SVDD: aprender una frontera de dominio

De una analogía entre palabras a una región del espacio. SVDD (*Support Vector Data Description*) aprende una esfera que describe un conjunto de ejemplos.

Caso utilizado: sin holguras

$$\min_{\mathbf{a}, R \geq 0} R^2$$

$$\text{sujeto a } \|\mathbf{z}_i - \mathbf{a}\|_2^2 \leq R^2 \quad \forall i.$$

$\mathbf{z}_i$: coordenadas PCA del término.
$\mathbf{a}$: centro aprendido. R: radio.
Los vectores de soporte determinan la frontera.

Tres modelos independientes

- Hombre: hombre, padre, hermano, abuelo.
- Mujer: mujer, madre, hermana, abuela.
- Computador: computador, procesador, teclado, monitor.

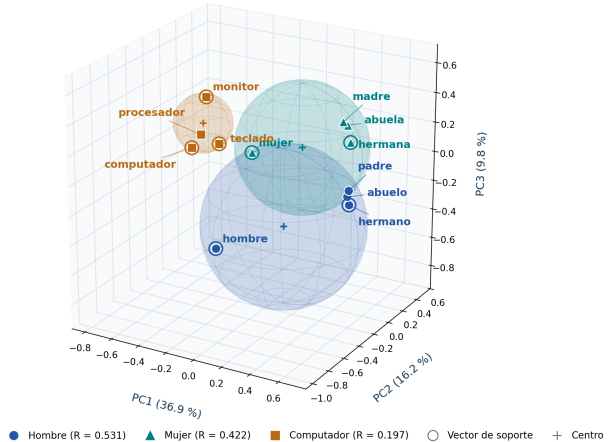
Un término se acepta si su distancia al centro no supera el radio.

Diseño del ejemplo: embeddings fastText normalizados L2, un PCA común a 3D y un SVDD lineal por grupo. Los dominios pueden solaparse.

Fuente: Tax y Duin (2004), Support Vector Data Description, Machine Learning 54, 45–66.

Fronteras de dominio: SVDD sobre PCA 3D

Tres modelos independientes · 4 palabras por grupo · esferas de mínimo radio sin holguras



SVDD entrenado en las coordenadas PCA 3D. PCA único sobre 12 embeddings normalizados L2; varianza conservada: 62.8%.

Cada esfera contiene los cuatro puntos de su grupo. Pueden solaparse: la pertenencia a los dominios es independiente.

Datos: fastText Wikipedia, español (wiki.es.vec). Método: SVDD lineal sin holguras, dual resuelto mediante SLSQP (SciPy).

La analogía como búsqueda: most_similar

Gensim sobre fastText en español

```
modelo.most_similar(
    positive=["rey", "mujer"],
    negative=["hombre"], topn=5
)
```

Gensim normaliza cada vector de entrada antes de componer la consulta.

$$\mathbf{r} = \widehat{\mathbf{v}}_{\text{rey}} - \widehat{\mathbf{v}}_{\text{hombre}} + \widehat{\mathbf{v}}_{\text{mujer}}.$$

Pos.	Palabra	Coseno
1	reina	0.6612
2	consorte	0.5856
3	desposa	0.5714
4	infanta	0.5498
5	esposa	0.5492

Reina: posición 1.
Vocabulario completo: 985 667 palabras.

Búsqueda y visualización

Los vecinos se buscan en las 300 dimensiones originales. Se excluyen rey, mujer y hombre. El PCA se utiliza solamente para visualizar.

Fuente: fastText Wikipedia (wiki.es.vec). Gensim 4.4.0, KeyedVectors.most_similar.

Embeddings de oraciones, pasajes y documentos

Unidad	Qué representa
Oración	Una expresión breve, a menudo con una idea principal.
Pasaje	Un fragmento que puede contener evidencia suficiente para responder.
Documento	Un texto completo que puede combinar varios temas.

La longitud de entrada importa

La salida puede tener dimensión fija aunque la entrada varíe en longitud. Cada modelo admite una longitud máxima de entrada y puede truncar el contenido.

La selección de la unidad de recuperación afecta qué información conserva el vector.

¿Cómo aprende el modelo la relación entre textos?

Ejemplo de entrenamiento para recuperación:

- Consulta q: «¿Cómo reparar el motor de un automóvil?»
- Positivo d^+: guía relevante para diagnosticar averías.
- Negativo fácil d_1^-: receta de pan.
- Negativo difícil d_2^-: catálogo de motores que no explica la reparación.

Objetivo intuitivo

Aumentar el score de (q, d^+) respecto a los scores de (q, d^-).

Los ejemplos positivos definen qué cuenta como útil. Los negativos difíciles permiten aprender distinciones más finas que el tema general.

Fuente: Lin et al., sección 5.4.2, entrenamiento de DPR y ANCE.

Aprendizaje contrastivo: una expresión del objetivo

Una pérdida habitual compara un positivo con un conjunto $\mathcal{N}_q$ de negativos:

$$\mathcal{L}_q = -\log \frac{\exp(s(q, d^+)/\tau)}{\exp(s(q, d^+)/\tau) + \sum_{d^- \in \mathcal{N}_q} \exp(s(q, d^-)/\tau)}.$$

- $s(q, d)$: score calculado a partir de los embeddings.
- $\tau > 0$: temperatura que regula la concentración de la distribución.
- El entrenamiento ajusta parámetros para reducir $\mathcal{L}_q$.

Es un ejemplo de objetivo, no una fórmula universal. Esta distribución de entrenamiento tampoco calibra automáticamente una probabilidad de relevancia en producción.

Fuente: Lin et al., sección 5.4.2. Formulación contrastiva con temperatura explícita.

Los datos delimitan lo aprendido

- Idiomas del entrenamiento.
- Dominios: conversación, ciencia, medicina.
- Tarea: paráfrasis o pregunta-respuesta.
- Calidad de positivos y negativos.

Un negativo mal etiquetado puede ser relevante. Un modelo de similitud puede favorecer textos temáticos que no responden.



Sentence-BERT y Sentence Transformers

- **Sentence-BERT:** adapta encoders Transformer mediante entrenamiento para obtener representaciones comparables de textos.
- **Sentence Transformers:** biblioteca y ecosistema para utilizar y entrenar modelos de embeddings, entre otras tareas.
- La calidad depende del modelo concreto y de su objetivo, no únicamente de utilizar un Transformer.

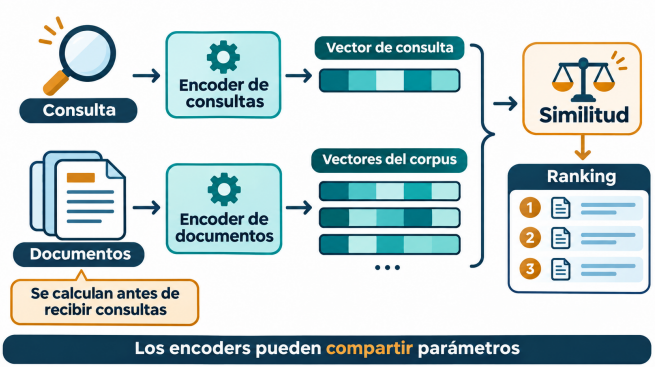
Qué revisar antes de usar un modelo

Idioma, tarea de entrenamiento, longitud máxima, dimensión, función de similitud y formato requerido para consultas y documentos.

Fuente: Lin et al., sección 5.4.1.

Bi-encoder: representar por separado

$$\mathbf{z}_q = f_{\theta_q}(q), \qquad \mathbf{z}_d = g_{\theta_d}(d), \qquad s(q,d) = \mathrm{sim}(\mathbf{z}_q, \mathbf{z}_d).$$



La representación del documento se calcula sin conocer la consulta.

Fuente: Lin et al., sección 5.4, pp. 139–143.

Simetría de la tarea y roles de las entradas

Similitud textual

«Reparar un automóvil.»
«Arreglar un vehículo.»

Se comparan textos con roles semejantes. Intercambiarlos no cambia la similitud buscada.

Recuperación asimétrica

Consulta: «¿Por qué se recalienta el motor?»
Documento: explicación técnica extensa.

Los roles y las longitudes pueden diferir.

Una tarea asimétrica puede usar el mismo encoder con prefijos distintos o encoders diferentes. Debe seguirse el formato de entrenamiento del modelo.

Fuente: Lin et al., secciones 5.4.1 y 5.4.2.

Similitud coseno: una fórmula que ya conocemos

Para vectores no nulos $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{j=1}^{m} u_j v_j}{\sqrt{\sum_{j=1}^{m} u_j^2} \sqrt{\sum_{j=1}^{m} v_j^2}}.$$

- Compara direcciones y elimina el efecto de la magnitud.
- Toma valores entre -1 y 1.
- Los embeddings densos pueden contener valores negativos.

Un coseno alto es evidencia del criterio aprendido. Su valor no equivale a un porcentaje de relevancia.

Producto punto y normalización L2

$$\mathbf{u} \cdot \mathbf{v} = \sum_{j=1}^{m} u_j v_j, \qquad \widehat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|_2}.$$

Equivalencia después de normalizar ambos vectores

$$\widehat{\mathbf{u}} \cdot \widehat{\mathbf{v}} = \cos(\mathbf{u}, \mathbf{v}).$$

- Sin normalización, el producto punto también depende de las magnitudes.
- Para vectores unitarios: $\|\widehat{\mathbf{u}} - \widehat{\mathbf{v}}\|_2^2 = 2 - 2\cos(\mathbf{u}, \mathbf{v})$.
- La función de scoring debe ser coherente con el entrenamiento.

Ejemplo numérico: calcular un ranking

Vectores inventados para practicar, no salidas de un modelo.

$$\mathbf{q} = (1, 2), \quad \mathbf{d}_1 = (2, 3), \quad \mathbf{d}_2 = (3, 0), \quad \mathbf{d}_3 = (-1, -2).$$

$$\|\mathbf{q}\| = \sqrt{5}, \quad \|\mathbf{d}_1\| = \sqrt{13}, \quad \|\mathbf{d}_2\| = 3, \quad \|\mathbf{d}_3\| = \sqrt{5}.$$

Documento	$\mathbf{q} \cdot \mathbf{d}$	Coseno	Posición
d_1	8	$8/\sqrt{65} \approx 0.992$	1
d_2	3	$1/\sqrt{5} \approx 0.447$	2
d_3	-5	-1	3

Ranking: d_1, d_2, d_3. El top-2 contiene d_1 y d_2.

Actividad breve: ¿puede cambiar el orden?

Mantengamos $\mathbf{q} = (1, 2)$ y comparemos:

$$\mathbf{d}_A = (2, 3), \qquad \mathbf{d}_B = (30, 0).$$

1. Calculen ambos productos punto.
2. Calculen ambos cosenos. Pueden utilizar la calculadora.
3. Expliquen por qué las dos funciones producen órdenes distintos.

Comprobación

Producto punto: 8 frente a 30; gana d_B.
Coseno: aproximadamente 0.992 frente a 0.447; gana d_A.

Multiplicar un vector por un escalar positivo cambia su magnitud, pero conserva su dirección.

Construcción del ranking para un corpus

Con N documentos y embeddings de dimensión m:

$$Z = \begin{bmatrix} \mathbf{z}_{d_1}^{\mathsf{T}} \\ \vdots \\ \mathbf{z}_{d_N}^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{N \times m}, \qquad \mathbf{s} = Z\mathbf{z}_q \in \mathbb{R}^N.$$

- Si filas y consulta tienen norma uno, $\mathbf{s}$ contiene cosenos.
- Ordenar de mayor a menor y seleccionar los k primeros.
- Cada fila debe conservar su asociación con el ID y texto del documento.

Primera implementación

Comparar con todo el corpus: búsqueda exhaustiva. El cálculo de scores cuesta $O(Nm)$ por consulta.

Qué se calcula antes y qué se calcula al consultar

Preparación del corpus

1. Definir textos e identificadores.
2. Aplicar el formato del modelo.
3. Calcular embeddings por lotes.
4. Guardar vectores y metadatos.

Atención de una consulta

1. Aplicar su formato de entrada.
2. Obtener el embedding.
3. Calcular scores.
4. Ordenar y devolver el top-k.

Si cambia el modelo o su configuración, los vectores almacenados deben seguir siendo compatibles con las nuevas consultas.

Preparar textos para un encoder neuronal

- Conservar el texto legible y utilizar el tokenizador del modelo.
- Evitar aplicar automáticamente stemming o eliminar stopwords.
- Respetar idioma, instrucciones y prefijos requeridos.
- Revisar truncamiento y longitud máxima en tokens.

Por qué importa conservar información

«Se recomienda usar este medicamento.»
«**No** se recomienda usar este medicamento.»

Eliminar ciertas palabras puede alterar el significado que necesita representar el encoder.

Para comparar modelos, conservar las mismas unidades documentales e IDs, aunque cada método tenga su propio preprocesamiento.

Similitud temática y relevancia

Consulta: «¿Cómo reparar el motor de un automóvil?»

- La guía de averías puede responder a la necesidad.
- El catálogo de motores comparte tema y vocabulario.
- El tutorial de bicicletas comparte una acción, pero cambia el objeto.

La relevancia depende de la necesidad de información

Un embedding puede acercar textos por relaciones que no bastan para resolver la consulta.

Para discutir

¿Cómo comprobaríamos que el primer resultado realmente ayuda al usuario?

Casos que requieren inspección cuidadosa

Caso	Posible error
Nombres y códigos	Confundir referencias cercanas: «A16» y «A100».
Cantidades y fechas	Acercar textos que difieren en un número decisivo.
Negación	Puntuar alto una afirmación y su negación.
Dominio especializado	Representar mal conceptos poco cubiertos en entrenamiento.
Textos extensos	Perder evidencia por truncamiento o por mezclar varios temas.

Implicación experimental

Diseñar consultas que comprueben estas distinciones. BM25 puede aportar evidencia léxica útil en algunos de estos casos.

Cómo interpretar un score

- Un coseno de 0.80 no significa «80% de relevancia».
- Los valores dependen del modelo y de su distribución de embeddings.
- Un umbral universal, por ejemplo 0.70, puede fallar entre dominios o modelos.
- Recuperar top-k devuelve candidatos incluso cuando ninguno responde bien.

Decisiones que requieren evidencia

Los umbrales de aceptación y la elección de k deben comprobarse con consultas y juicios representativos del uso previsto.

No comparar directamente scores de BM25 y coseno como si tuvieran la misma escala.

Puente hacia la clase de evaluación

Comparación que realizaremos

TF-IDF con coseno, BM25 y recuperación densa con embeddings.

- Mismas consultas y unidades de recuperación.
- Juicios de relevancia explícitos e independientes del score.
- Métricas para documentos recuperados y para el orden del ranking.
- Análisis de errores y del costo de recuperación.

Para discutir

Un sistema recupera más documentos relevantes, pero los coloca más abajo. ¿Es mejor?

La siguiente clase formalizará esta comparación con Precision, Recall y métricas de ranking.

Ideas que deben quedar claras

- Un embedding representa un texto mediante un vector aprendido.
- Su geometría depende del entrenamiento y de la función de comparación.
- Un bi-encoder permite precalcular los vectores del corpus.
- Los scores producen un ranking, pero requieren evaluación para demostrar utilidad.

Comprobación conceptual

¿Por qué dos textos sin palabras compartidas pueden tener un coseno alto?

¿Por qué ese coseno alto puede ser insuficiente para considerarlos relevantes?

Bibliografía y lecturas

- **Manning, Raghavan y Schütze.** *Introduction to Information Retrieval*. Cap. 6: representación vectorial y coseno. Caps. 11–12: modelos probabilísticos y de lenguaje. Cap. 18: LSI.
- **Baeza-Yates y Ribeiro-Neto.** *Modern Information Retrieval* (1999). Cap. 2: modelos de recuperación y taxonomía clásica.
- **Lin, Nogueira y Yates.** *Pretrained Transformers for Text Ranking: BERT and Beyond*. Sección 2: Transformers. Sección 5.1: recuperación densa. Sección 5.4.1: Sentence-BERT. Sección 5.4.2: DPR y ANCE.

Complemento: Tax y Duin (2004). *Support Vector Data Description*. Machine Learning, 54, 45–66.

Lectura principal para esta clase

Lin et al., sección 5.4, especialmente pp. 139–145 de la numeración impresa del texto proporcionado.